

Top Data Engineer Interview Questions

(TCS / Infosys / Wipro – 2025)

Databricks & PySpark

Q1□ How to flatten nested JSON in Spark?

- Use `from_json` + `explode` or `selectExpr` with dot notation

```
from pyspark.sql.functions import explode, col
df = spark.read.json("file.json")
flat_df = df.selectExpr("id", "nested.field1 as field1", "nested.field2 as field2")
```

Q2□ How to handle missing or null values?

- `dropna()`, `fillna()`, `filter()`

```
df.dropna()
df.fillna({'col1': 'default', 'col2': 0})
df.filter(df.col1.isNotNull())
```

Q3□ Write a UDF to return only first name from full name.

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

def first_name(fullname):
    return fullname.split()[0]

first_name_udf = udf(first_name, StringType())
df = df.withColumn("first_name", first_name_udf(df.full_name))
```

Q4□ Compute 7-day rolling average of stock closing prices.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import avg

window = Window.partitionBy("company").orderBy("date").rowsBetween(-6, 0)
df = df.withColumn("7_day_avg", avg("closing_price").over(window))
```

Q5□ Repartition vs Coalesce

- **Repartition:** Full shuffle, increases/decreases partitions, ensures even distribution.
 - **Coalesce:** Merges partitions (reduces count) without shuffle — faster for downsizing.
-

Q6□ Spark Architecture - Jobs, Stages, Tasks

- **Driver:** Coordinates execution.
 - **Executors:** Run tasks.
 - **Job:** Triggered by an action.
 - **Stage:** Unit between shuffles.
 - **Task:** Runs per data partition.
-

Q7□ inferSchema vs manual schema?

inferSchema → 2 jobs (schema inference + read)

manual schema → 1 job → faster!

Q8□ Query Optimization in PySpark/Spark SQL

- Cache reused data
 - Use broadcast joins
 - Reduce shuffles
 - Partition/bucket
 - Avoid wide transformations
-

Q9□ AQE (Adaptive Query Execution)

Optimizes at runtime — merges shuffle partitions, adjusts joins, improves performance.

Q10 What is Liquid Clustering (Databricks)?

Organizes Delta Lake data via **Z-order columns** → boosts selective query performance.

Azure Synapse + ADF

Q11□ Databricks vs Synapse

- Databricks → Spark-based for ETL, streaming, ML.
 - Synapse → SQL-based for analytics + BI (Power BI friendly).
-

Q12□ Managed vs External Table

- **Managed:** Synapse owns data + metadata.
- **External:** Only metadata; data in external storage.

Q13 □ SCD Type 1 vs Type 2

- Type 1 → Overwrites data (no history).
- Type 2 → Inserts new record (keeps history).

Q14 □ Where to apply data quality checks?

- At ingestion layer (Bronze)
- Schema validation
- Nulls & duplicates check
- Data type validation

Q15 □ How to handle error records?

- Redirect to **staging/error table / DLQ**,
- Log & alert via ADF monitor.

Azure Storage & Access

Q16 □ ADLS Gen1 vs Gen2

- Gen2 → Hierarchical namespace, better performance, cheaper.
- Gen1 → Legacy, flat namespace.

Q17 □ Disaster Recovery (ADF/Databricks)

- Geo-redundant storage
- Multi-region deployment
- Pipeline backups
- Automated failover

Q18 □ Configure Unity Catalog (Databricks)

- Premium workspace
 - Access connector to storage
 - Create metastore + schema
 - Grant RBAC permissions
-

Q19 ☐ Schema-level access control

```
GRANT SELECT ON SCHEMA sales TO ROLE data_analyst;
```

Q20 ☐ Access Tokens in Databricks

User Settings → Access Tokens tab

OR via Entra ID / PAT for API.

Q21 ☐ Can Unity Catalog use multiple metastores?

- Yes — 1 per workspace/environment.

SQL Essentials

Q22 ☐ Find duplicate rows.

```
SELECT col1, col2, COUNT(*)
FROM table_name
GROUP BY col1, col2
HAVING COUNT(*) > 1;
```

Q23 ☐ Third highest salary (no window funcs).

```
SELECT MAX(salary)
FROM table_name
WHERE salary < (
    SELECT MAX(salary) FROM table_name
    WHERE salary < (SELECT MAX(salary) FROM table_name)
);
```

Q24 ☐ Top 10 active users (PySpark)

```
from pyspark.sql.functions import count
df.groupBy("user_id").agg(count("*").alias("activity_count"))\
    .orderBy("activity_count", ascending=False).limit(10)
```

Q25 ☐ Difference - DataFrame SQL vs T-SQL

- **DataFrame SQL:** Distributed, Spark syntax.
 - **T-SQL:** Centralized, SQL Server specific.
-

Q26 ☐ Z-ordering in Delta Tables

Clusters data on key columns to minimize scan and improve read performance